tempssm: State-space modeling for temperature time series in R

Akira Hirao

2026-05-13

Summary

R package **tempssm** provides tools for state-space analysis of temperature time series, focusing on linear Gaussian state-space models estimated via Kalman filtering and smoothing as implemented in the KFAS package (Helske, 2017).

Key features

- Designed for temperature time series with arbitrary seasonal frequencies; currently validated primarily on monthly data
- Estimates latent states using linear Gaussian state-space models combined with Kalman filtering and smoothing
- Models temperature dynamics as a sum of interpretable latent components, including a long-term trend, seasonal cycle, autoregressive structure, and optional exogenous effects
- Allows users to specify an arbitrary order of the autoregressive component (default: AR(1))
- Implements time-series cross-validation for model evaluation

Sate-space model in tempssm

We consider an extended version of the Basic Structural Time Series Model (BSTSM) to describe temperature time series with explicit long-term trend, seasonal variability, and auto-regressive component. The observation equation is given by

$$y_t = \alpha_t + v_t. \tag{1}$$

where t denotes the time index, y_t is the observed temperature, α_t is the latent state, and v_t is the observation error.

The latent state is decomposed as

$$\alpha_t = \mu_t + s_t + r_t + E_t. \tag{2}$$

where μ_t is the level (trend), s_t the seasonal component, r_t a stationary autoregressive component, and E_t the contribution of exogenous variables.

The level component follows a second-order stochastic process:

$$\mu_t = 2\mu_{t-1} - \mu_{t-2} + \zeta_t, \qquad \zeta_t \sim \mathcal{N}(0, \sigma_\zeta^2). \tag{3}$$

where ζ_t is the process error of the level component.

The seasonal component is modeled with a sum-to-zero constraint:

$$s_t = -\sum_{i=t-f}^{t-1} s_i + \omega_t, \qquad \omega_t \sim \mathcal{N}(0, \sigma_\omega^2). \tag{4}$$

where ω_t is the process error of the seasonal component and f denotes the seasonal frequency (e.g., $f = 12$ for monthly data). This formulation ensures that seasonal effects are identifiable and comparable across

different temporal resolutions. By enforcing the sum-to-zero constraint over one full seasonal cycle, the seasonal component captures recurring deviations from the underlying trend without introducing long-term drift. In models without a seasonal component, the seasonal term s_t is set to zero and omitted from the satate equation.

The autoregressive component follows an lth-order autoregressive (AR) process:

$$r_t = \phi_1 r_{t-1} + \phi_2 r_{t-2} + \cdots + \phi_l r_{t-l} + \tau_t, \qquad \tau_t \sim \mathcal{N}(0, \sigma_\tau^2). \tag{5}$$

Here, τ_t denotes the process error of the autoregressive component. The process error terms (ζ_t, ω_t, and τ_t) are assumed to be mutually independent and normally distributed.

The exogenous component is defined as

$$E_t = \beta_1 x_{1,t} + \beta_2 x_{2,t} + \cdots + \beta_m x_{m,t}. \tag{6}$$

The exogenous term E_t represents the influence of external factors, which may include large-scale climate indices, regional environmental variables, or other physically motivated predictors relevant to the observed temperature time series. Each exogenous variable enters the model linearly through a time-invariant regression coefficient ($\beta_1, \beta_2, \ldots, \beta_m$), allowing both the magnitude and direction of its contribution to be estimated jointly with the latent state components.

When applying the model to observational data, the total number of parameters to be estimated (k) depends on the order of autoregressive component and the number of exogenous variables included. For example, in a model with a second-order autoregressive component and no exogenous covariates, six parameters are estimated: the observation error variance, the process error variances associated with the level, seasonal, and autoregressive components, and the first- and second-order autoregressive coefficients (ϕ_1 and ϕ_2). In the general case with an l-order autoregressive component and m exogenous variables, a total number of parameters is $k = 4 + l + m$.

The core implementation of parameter estimation procedure is based on the supplementary code provided by Baba et al. (2024). Parameter estimation is performed using a two-step optimization strategy recommended by Helske (2017). In the first step, model parameters are estimated using the Nelder-Mead method (Nelder & Mead, 1965) with user-specified initial values. The resulting estimates are then used as initial values in a second optimization step based on the BFGS algorithm (Shanno, 1970). The main model-fitting function, `tempssm()`, returns both filtering and smoothing estimates. Unless otherwise stated, all results presented in this vignette are based on the smoothed estimates.

How to use

Set Ennvironment

Load the following libraries for executing 'How to use'.

```
## Set libraries
library(tempssm)
library(forecast)

library(purrr)
library(tibble)
library(readr)
library(dplyr)
library(ggplot2)

library(patchwork)

# For parallel processing
```

```r
library(progressr)
library(future.apply)
library(future)

# enable parallel execution (optional)
if (interactive()) {
  plan(multisession)
} else {
  plan(sequential)
}
```

Input Data Format

Input data for **tempssm** must be supplied as an R `ts` object, which represents a regularly spaced time series (see `?stats::ts` or https://stat.ethz.ch/R-manual/R-devel/library/stats/html/ts.html).

To support data preparation, the package includes utility functions that convert external observational data into `ts` objects suitable for model fitting (see Appendix).

Practice I: Applying State-Space Modelto a Univariate Temperature Time Series

Objective

The objective of this practice is to demonstrate the basic application of a linear Gaussian state-space model to a univariate temperature time series. This example serves as an introduction to the modeling framework and highlights the role of autoregressive dynamics without the inclusion of exogenous variables.

Loading the Dataset

A sample sea surface temperature (SST) dataset is included in the package.

- **Dataset**: Monthly sea surface temperature (SST) off Niigata, Japan
- **Unit**: Degrees Celsius
- **Period**: February 2002 to December 2023
  “

This dataset is derived from observations archived at Japan Oceanographic Data Center (JODC), Hydrographic and Oceanographic Department, Japan Coast Guard. Original daily SST data were obtained from https://www.jodc.go.jp/jodcweb/JDOSS/index.html and aggregated into monthly means.

```r
data(niigata_sst) # load a ts object of SST off Niigata
head(niigata_sst)
```

```
##            Jan       Feb       Mar       Apr       May       Jun
## 2002  9.951613  8.332143  9.348387 11.713333 14.529032 18.906667
```

```r
summary(niigata_sst)
```

```
##       Temp
##  Min.   : 7.707
##  1st Qu.:11.217
##  Median :16.345
##  Mean   :17.033
##  3rd Qu.:22.787
##  Max.   :28.897
```

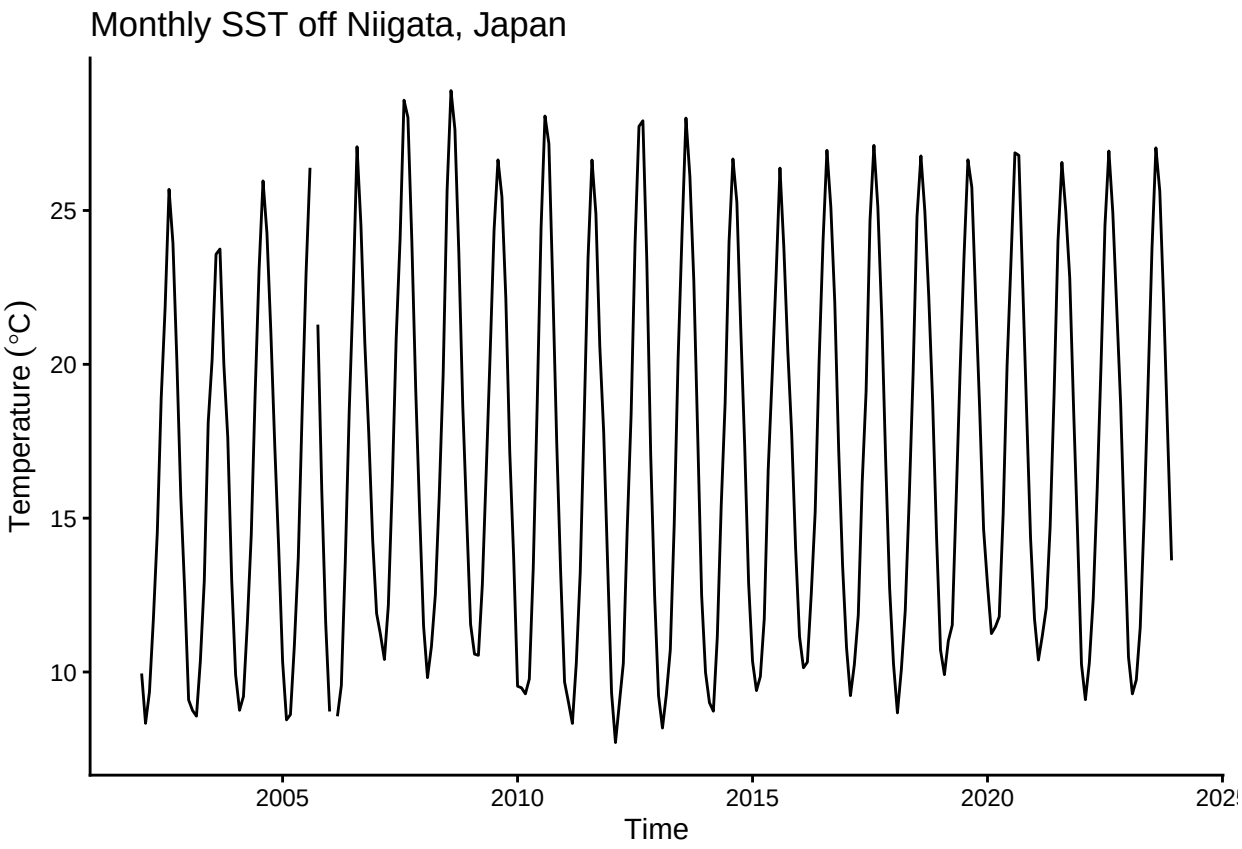
```
##  NA's   :2
```

The dataset includes two missing observations. Even if missing observations was in your dataset included, there are retained and handled explicitly within the state-space modeling framework.

Plotting the Monthly SST Time Series

We begin by visualizing the monthly SST time series to examine its overall structure, including apparent trends, seasonal variability, and missing observations.

```
plt_niigata_sst <- forecast::autoplot(niigata_sst) +
  labs(y = expression(Temperature~(degree*C)),
       x = "Time") +
  ggtitle("Monthly SST off Niigata, Japan") +
  theme_classic()

plot(plt_niigata_sst)
```



Applying a Linear Gaussian State-Space Model

Here, we apply linear Gaussian state-space models to the univariate SST time series and examine the effect of the autoregressive (AR) order on model behavior.
Specifically, three models are fitted with the order of the autoregressive component varying from 1 to 3, while all other model components, including the explicit seasonal cycle, are kept the same.

By comparing models with different AR orders, we assess how short- and longer-term temporal dependencies are represented within the state-space framework.
The model summaries provide parameter estimates and diagnostics that can be used to evaluate the adequacy of different autoregressive orders. Model comparison and selection are discussed in the following section.

```r
# model with first-order autoregressive component
res <- tempssm(niigata_sst) # first order of auto-regressive model (ar_order=1: default)
summary(res)
```

```
## tempssm summary
## -----------------
## Call:
## tempssm(temp_data = niigata_sst)
##
## Model fit:
##   Log-likelihood : -277.95
##   k              : 5
##   AIC            : 565.9
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.00598564
##   State (Q trend): 1.268118e-07
##   State (Q season): 0.001346139
##   State (Q ar): 0.4097882
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.7442999
```

```r
# model with second-order autoregressive component
res_ar2 <- tempssm(niigata_sst,ar_order=2)
summary(res_ar2)
```

```
## tempssm summary
## -----------------
## Call:
## tempssm(temp_data = niigata_sst, ar_order = 2)
##
## Model fit:
##   Log-likelihood : -277.95
##   k              : 6
##   AIC            : 567.89
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.04528328
##   State (Q trend): 1.411893e-07
##   State (Q season): 0.001338972
##   State (Q ar): 0.3463124
##
## Components of auto-regression:
##   Order of AR: 2
##   Coefficient of AR1: 0.8278959
##   Coefficient of AR2: -0.0651634
```

```r
# model with third-order autoregressive component
res_ar3 <- tempssm(niigata_sst,ar_order=3)
summary(res_ar3)
```

```
## tempssm summary
## -----------------
## Call:
## tempssm(temp_data = niigata_sst, ar_order = 3)
##
## Model fit:
##   Log-likelihood : -277.94
##   k              : 7
##   AIC            : 569.89
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.0002537319
##   State (Q trend): 1.418092e-07
##   State (Q season): 0.001336738
##   State (Q ar): 0.418686
##
## Components of auto-regression:
##   Order of AR: 3
##   Coefficient of AR1: 0.7335228
##   Coefficient of AR2: 0.0118387
##   Coefficient of AR3: -0.005368551
```

Model selection based on AIC

```
# Extract AIC
AIC(res)
```

```
## [1] 565.8955
```

```
AIC(res_ar2)
```

```
## [1] 567.8903
```

```
AIC(res_ar3)
```

```
## [1] 569.8889
```

```
AIC_table_res <- tibble(model=c("AR1","AR2","AR3"),
                        AIC = c(AIC(res),AIC(res_ar2),AIC(res_ar3))) %>%
  mutate(delta_AIC = AIC - min(AIC))

AIC_table_res %>% knitr::kable()
```

model	AIC	delta_AIC
AR1	565.8955	0.000000
AR2	567.8903	1.994836
AR3	569.8889	3.993417

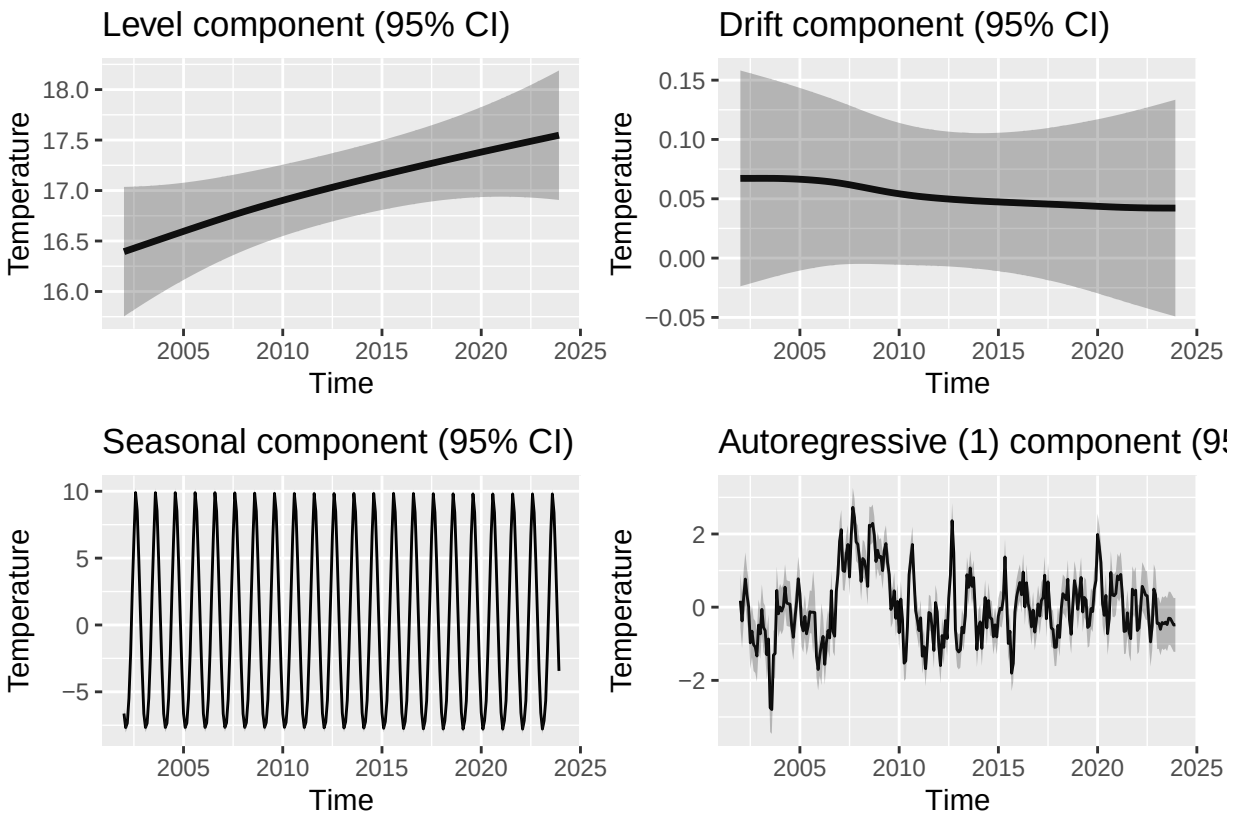
AR1 model is better than the other models.

Plotting Level, Drift, Seasonal, and Auto-Regressive Components

We visualize the estimated long-term evolution of temperature levels and their rates of change (drift) by extracting the corresponding latent components from the state-space model.Additionally, seasonal variability

and autoregressive dependence are plotted out, allowing the underlying trend behavior to be examined more clearly.

```
# plot each of components at once
plt_level <- autoplot(res, component = c("level"))
plt_drift <- autoplot(res, component = c("drift"))
plt_season <- autoplot(res, component = c("season"))
plt_ar1 <- autoplot(res, component = c("ar1"))

plt_level + plt_drift + plt_season + plt_ar1 + patchwork::plot_layout(ncol=2, nrow=2)
```



The level component shows a persistent upward trend in sea surface temperature over the study period, while the drift component indicates a relatively stable positive rate of change. The shaded gray areas represent 95% confidence intervals for the estimated latent states, illustrating the uncertainty associated with each of estimated components.

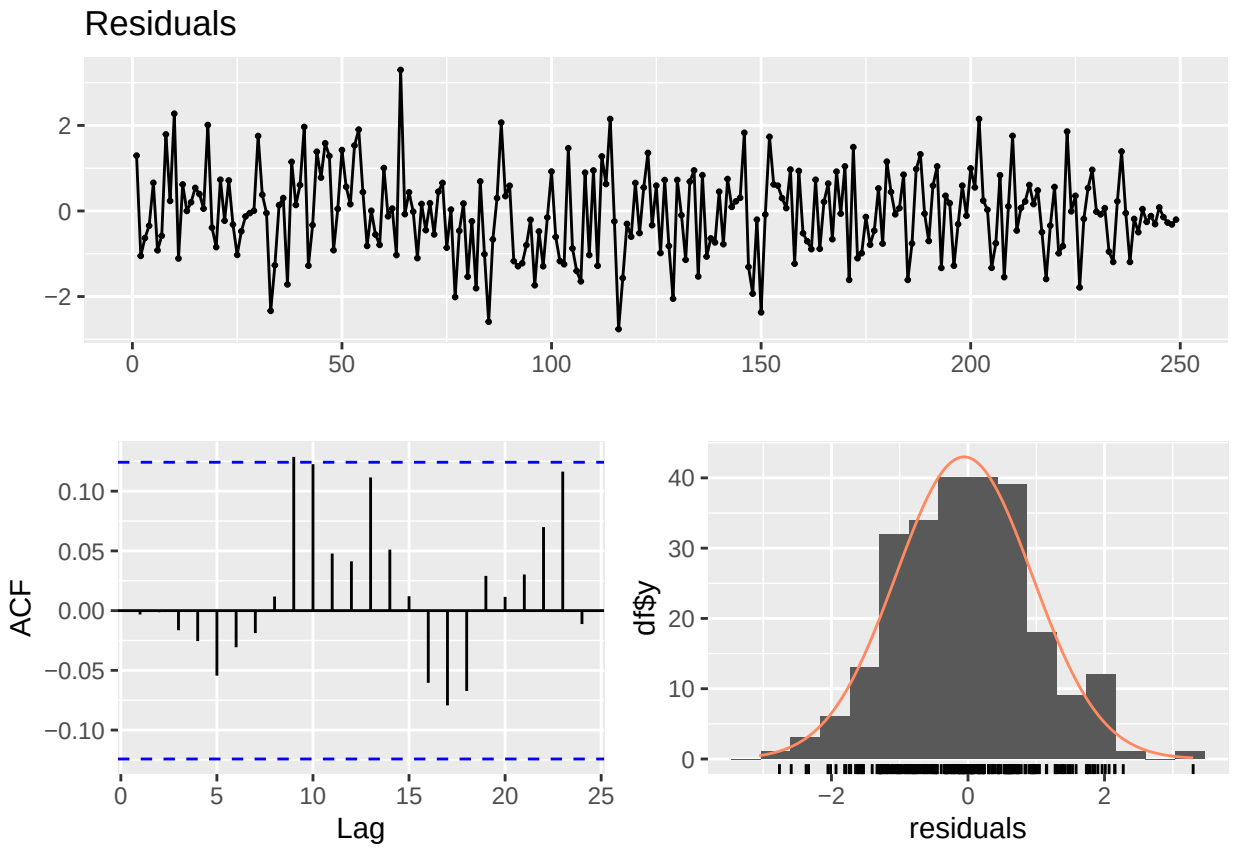
To visualize all components simultaneously, you can use `autoplot(res)`, which provides a convenient overview of the model decomposition.

Simple Model Diagnostics

```
diag <- diagnose_residuals(res)
print(diag)
```

```
## # A tibble: 1 x 4
##   lb_stat lb_df lb_pvalue kurtosis
##     <dbl> <dbl>     <dbl>    <dbl>
## 1 0.00312     2     0.998     3.06
```

```r
plot_residual_diagnostics(res)
```



Autocorrelation of residuals was not significant by Ljung-Box test ($P > 0.05$).

Estimated Parameters and Components

```r
# Smoothing estimates
alpha_hat <- res_ar2$kfs$alphahat
head(alpha_hat)
```

```
##              level       slope sea_dummy1 sea_dummy2 sea_dummy3 sea_dummy4
## Jan 2002 16.38745 0.005741462  -6.624146  -3.337748  0.6067547  4.7591913
## Feb 2002 16.39319 0.005741621  -7.677456  -6.624146 -3.3377475  0.6067547
## Mar 2002 16.39893 0.005741616  -7.345735  -7.677456 -6.6241456 -3.3377475
## Apr 2002 16.40467 0.005741502  -5.477243  -7.345735 -7.6774564 -6.6241456
## May 2002 16.41042 0.005741531  -2.216006  -5.477243 -7.3457349 -7.6774564
## Jun 2002 16.41616 0.005741682   2.467139  -2.216006 -5.4772431 -7.3457349
##           sea_dummy5 sea_dummy6 sea_dummy7 sea_dummy8 sea_dummy9 sea_dummy10
## Jan 2002  8.5283425  9.9003246  6.4165810  2.4671389  -2.216006   -5.477243
## Feb 2002  4.7591913  8.5283425  9.9003246  6.4165810   2.467139   -2.216006
## Mar 2002  0.6067547  4.7591913  8.5283425  9.9003246   6.416581    2.467139
## Apr 2002 -3.3377475  0.6067547  4.7591913  8.5283425   9.900325    6.416581
## May 2002 -6.6241456 -3.3377475  0.6067547  4.7591913   8.528342    9.900325
## Jun 2002 -7.6774564 -6.6241456 -3.3377475  0.6067547   4.759191    8.528342
##           sea_dummy11      arima1       arima2
## Jan 2002   -7.345735  0.13755973 -0.008613188
## Feb 2002   -5.477243 -0.28072777 -0.008963859
```

```
## Mar 2002   -2.216006  0.27844430  0.018293176
## Apr 2002    2.467139  0.70444118 -0.018144377
## May 2002    6.416581  0.34181822 -0.045903781
## Jun 2002    9.900325 -0.03076938 -0.022274037
```

```r
# Smoothing estimate of level component
level_ts <- get_level_ts(res_ar2)

# Smoothing estimate of drift component
drift_ts <- get_drift_ts(res_ar2)

# Average drift rate per year across the full period
mean_drift_year <- mean(drift_ts)
print(mean_drift_year)
```

```
## [1] 0.05271973
```

Average annual increase in SST is approximately 0.05 °C.

Practice II: Applying State-Space Model to a Temperature Time Series with an Exogenous Variable

Objective

The objective of this practice is to investigate and quantify the influence of a large-scale climate mode on temperature variations. Specifically, we examine the effect of the Pacific Decadal Oscillation (PDO) as an exogenous variable on SST observed off Yamaguchi Prefecture, Japan, within a state-space modeling framework.

Loading the Dataset 1: SST

A sample sea surface temperature (SST) dataset is included in the package.

- **Dataset**: Monthly sea surface temperature (SST) off Yamaguchi Prefecture, Japan
- **Unit**: Degrees Celsius
- **Period**: February 2002 to December 2023
  “

This dataset is derived from observations archived at Japan Meteorological Agency (JMA). Data obtained from the JMA website: https://www.jma.go.jp/jma/indexe.html

```r
data(yamaguchi_sst) # load a ts object of SST off Niigata
head(yamaguchi_sst)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1982 14.85516 13.36500 13.55645 14.29933 17.72419 21.53000
```

Loading Dataset 2: PDO Index as an Exogenous Variable

- **Data**: Monthly Pacific Decadal Oscillation (PDO) index (JMA)
- **Period**: January 1901 to December 2025

The Pacific Decadal Oscillation (PDO) index is defined as the projection of monthly mean sea surface temperature (SST) anomalies onto the leading empirical orthogonal function (EOF) of SST variability over the North Pacific north of 20°N. The EOF is computed using SST anomalies for 1901–2000, defined relative

to the 1901–2000 monthly climatology. To remove the global warming signal, the global-mean SST anomaly is subtracted from each grid point prior to the EOF analysis. In this package, we use the PDO index provided by the Japan Meteorological Agency (JMA), available at https://www.data.jma.go.jp/kaiyou/data/shindan/b_1/pdo/pdo.txt.

```
data(pdo) # load a ts object of NAO index
head(pdo)
```

```
##          Jan     Feb     Mar     Apr     May     Jun
## 1901  1.0040  0.7403  0.9011 -0.0109 -0.2325 -0.6810
```

Intersecting the Temperature and PDO Time Series

For state-space modeling with exogenous variables, all input time series must share a common and aligned time index. In this step, the temperature and PDO time series are restricted to their overlapping period by trimming the leading and trailing portions, ensuring that both datasets cover an identical time span.

The function `tempssm::trim_ts_overlap()` is used to align the two `ts` objects on a shared timeline, returning a multivariate time series containing only the common period.

```
# Generate an object on a shared timeline
yamaguchi_sst_trim <- trim_ts_overlap(yamaguchi_sst,
                                      pdo,
                                      temp_name = "Temp",
                                      exo_name="PDO")$temperature


pdo_trim <- trim_ts_overlap(yamaguchi_sst,
                            pdo,
                            temp_name = "Temp",
                            exo_name="PDO")$exogenous

start(yamaguchi_sst_trim)
```

```
## [1] 1982    1
```

```
end(yamaguchi_sst_trim)
```

```
## [1] 2025   12
```

```
start(pdo_trim)
```

```
## [1] 1982    1
```

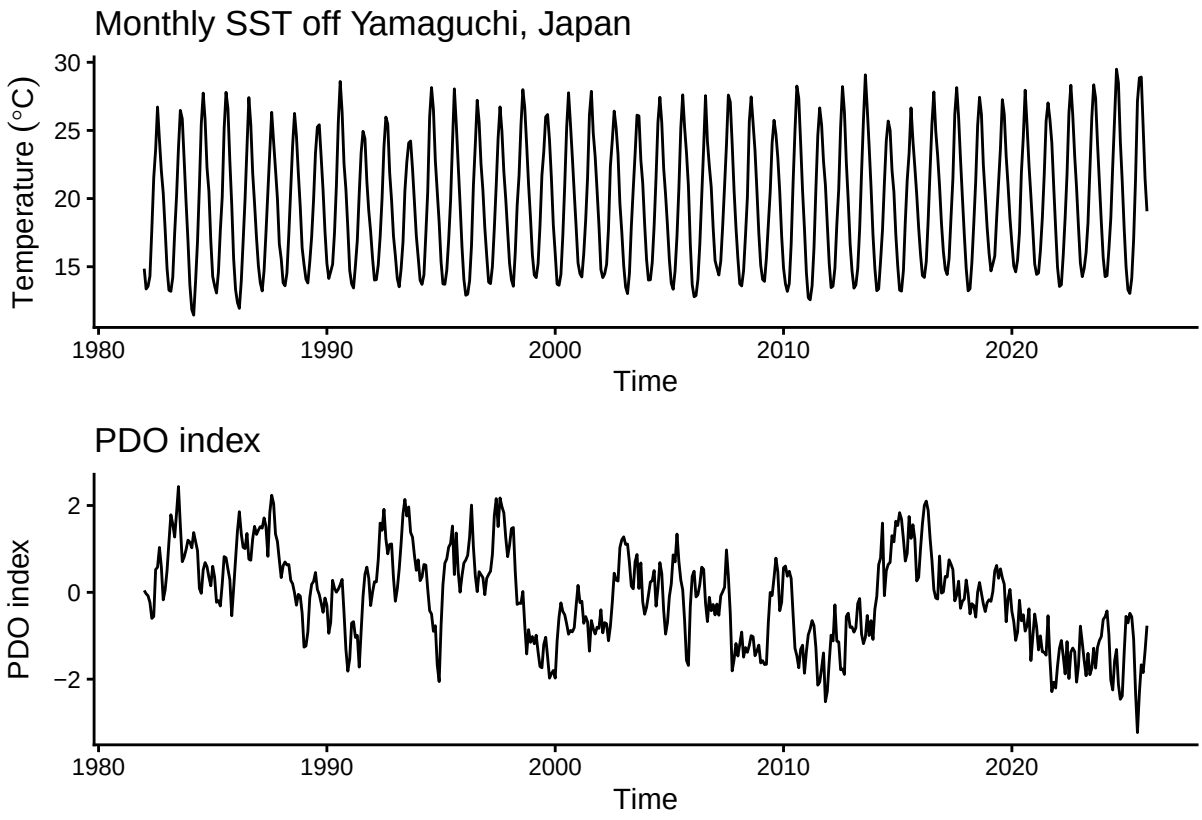
```
end(pdo_trim)
```

```
## [1] 2025   12
```

Plotting Time Series of Air Temperature and NAO Index

```
plt_yamaguchi_sst_trim <- forecast::autoplot(yamaguchi_sst_trim) +
  labs(y = expression(Temperature~(degree*C)),
       x = "Time") +
  ggtitle("Monthly SST off Yamaguchi, Japan") +
  theme_classic()


plt_pdo <- forecast::autoplot(pdo_trim) +
```

```
  labs(x = "Time", y = "PDO index") +
  ggtitle("PDO index") +
  theme_classic()

plt_yamaguchi_sst_trim + plt_pdo + patchwork::plot_layout(ncol=1)
```



Applying a Model Without an Exogenous Variable

We first fit a baseline state-space model that does not include any exogenous variables. This model serves as a reference case in which temperature variability is explained solely by the latent trend, seasonal cycle, and autoregressive dependence.

```
res_without <- tempssm(yamaguchi_sst_trim,ar=2)
summary(res_without)
```

```
## tempssm summary
## -----------------
## Call:
## tempssm(temp_data = yamaguchi_sst_trim, ar_order = 2)
##
## Model fit:
##   Log-likelihood : -466.08
##   k              : 6
##   AIC            : 944.17
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.1207592
```

```
##   State (Q trend): 2.163326e-07
##   State (Q season): 2.910014e-13
##   State (Q ar): 0.1071189
##
## Components of auto-regression:
##   Order of AR: 2
##   Coefficient of AR1: 1.178246
##   Coefficient of AR2: -0.4720562
```

The fitted model converges successfully and includes second-order autoregressive [AR(2)] component. Prior testing of autoregressive orders from AR(1) to AR(3) indicated that AR(2) provided the best model fit for both the baseline model and the model including the exogenous PDO index. For brevity, the detailed results of this comparison are omitted here; interested users are encouraged to explore alternative AR orders within their own analyses.

This baseline model provides a useful benchmark for evaluating the additional explanatory power of the PDO index introduced in the following section.

Applying Model With an Exogenous Variable

```
res_with <- tempssm(temp_data = yamaguchi_sst_trim,
                    exo_data = pdo_trim,
                    ar_order = 2)
summary(res_with)
```

```
## tempssm summary
## -----------------
## Call:
## tempssm(temp_data = yamaguchi_sst_trim, exo_data = pdo_trim,
##     ar_order = 2)
##
## Model fit:
##   Log-likelihood : -427.33
##   k              : 7
##   AIC            : 868.65
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.05748997
##   State (Q trend): 3.864314e-19
##   State (Q season): 1.610093e-54
##   State (Q ar): 0.1792553
##
## Components of auto-regression:
##   Order of AR: 2
##   Coefficient of AR1: 0.8656252
##   Coefficient of AR2: -0.1615365
## Exogenous variable    PDO
## Estimated coefficient     -0.4406609
## Lower CI  -0.5296035
## Upper CI  -0.3517183
```

The estimated coefficient for the exogenous PDO index is negative (-0.44), and its 95% confidence interval does not include zero

$$-0.53, -0.35$$

, indicating a statistically significant relationship between local SST off Yamaguchi Prefecture and the PDO variability.

Specifically, the model suggests that a one-unit increase in the PDO index is associated with an average decrease of approximately 0.44 °C in monthly SST, after accounting for the underlying trend, seasonal cycle, and autoregressive dependence. This result implies that positive phases of the PDO contribute systematically to cooler temperature conditions at the study site.

Importantly, this effect is identified in addition to the internal dynamics of the temperature time series, rather than being an artifact of an improved representation of the trend or temporal dependence. Compared with the baseline model without exogenous variables, the statistical significance of the PDO coefficient indicates that the PDO acts as an independent large-scale climate driver influencing local temperature variability.

We next compare the overall goodness of fit between the two models using the Akaike Information Criterion (AIC).

Model comparison based on AICs

Model selection criteria such as the AIC provide a quantitative measure of model adequacy that balances goodness of fit against model complexity. Lower AIC values indicate a more parsimonious model with better support from the data.

```
AIC_without <- AIC(res_without)
AIC_with <- AIC(res_with)

models_AICs <- tibble(
  model = c("Without","With"),
  AIC = c(AIC_without,AIC_with),
  delta_AIC = min(AIC)-AIC
  )

models_AICs %>% knitr::kable()
```

model	AIC	delta_AIC
Without	944.1678	-75.51744
With	868.6503	0.00000

The model including the PDO index as an exogenous variable exhibits a substantially lower AIC than the baseline model without exogenous variables, indicating a markedly better overall fit. This result supports the conclusion that explicitly accounting for NAO variability improves the statistical description of the temperature time series beyond what is achieved by internal dynamics alone.

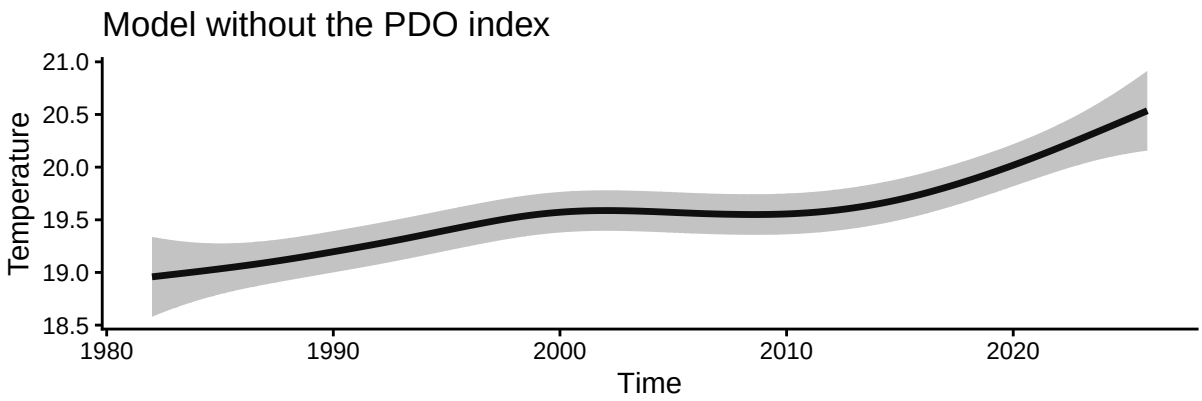
It should be noted that AIC-based comparisons are meaningful only among models fitted to the same time series over an identical period. Moreover, in state-space models, AIC differences reflect both regression effects and changes in the stochastic structure of latent components. Therefore, AIC should be used as a complementary diagnostic alongside parameter estimates and their uncertainty, rather than as a sole basis for inference.

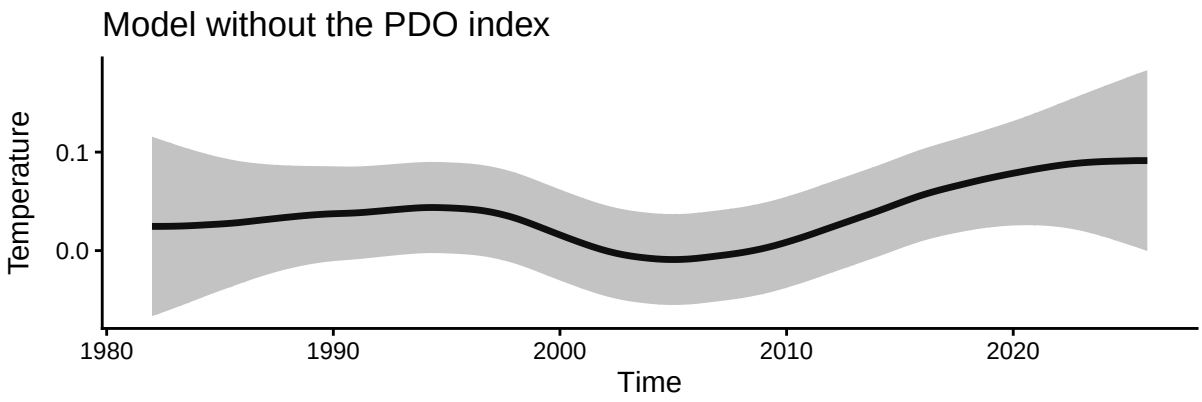
To further assess the robustness and predictive performance of the models, we employ time-series cross-validation as described bellow.

Plotting Level and Drift Components with 95% CI

```
plt_level_without_ts <- autoplot(res_without,
                                 component=c("level")
```

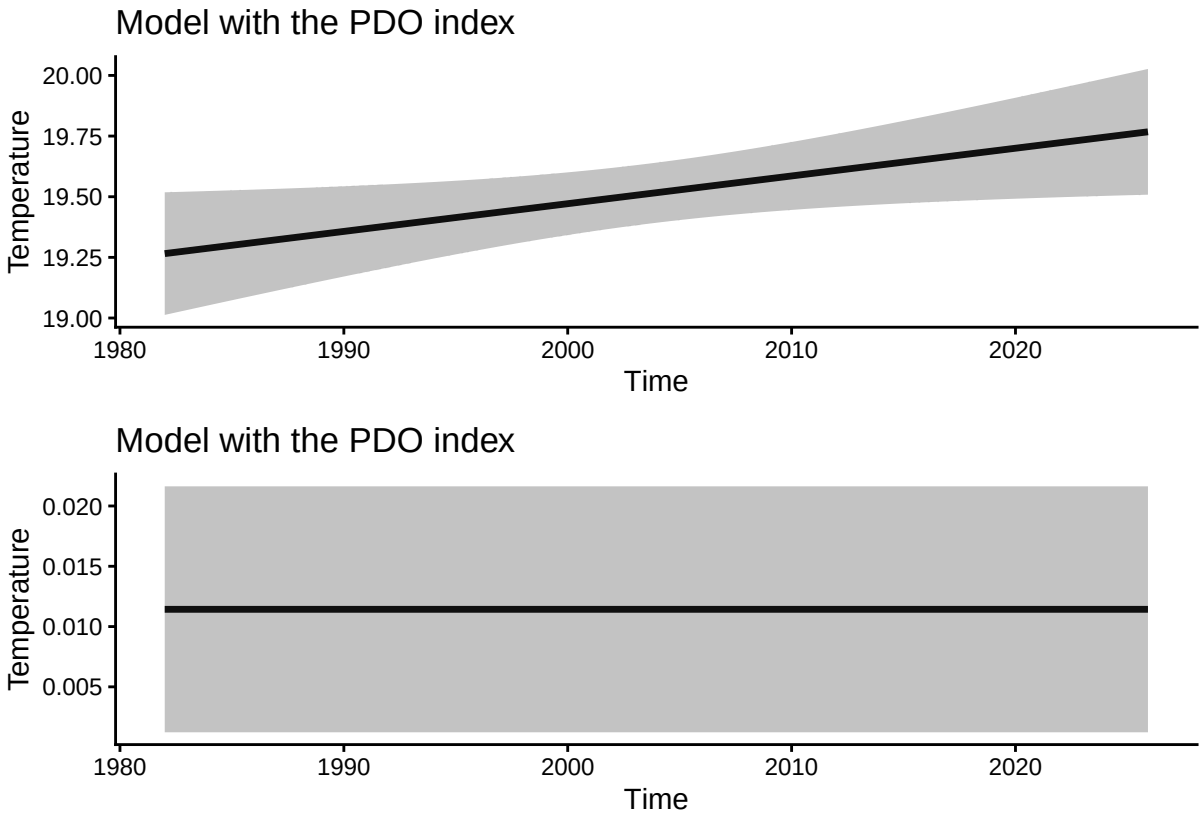
```
                                  ) +
  labs(title="Model without the PDO index") +
  theme_classic()

plt_drift_without_ts <- autoplot(res_without,
                                 component=c("drift")
                                 ) +
  labs(title="Model without the PDO index") +
  theme_classic()

plt_level_drift_without_ts <- plt_level_without_ts +
  plt_drift_without_ts +
  patchwork::plot_layout(ncol=1)

plot(plt_level_drift_without_ts)
```






```
plt_level_with_ts <- autoplot(res_with,
                              component=c("level")
                              )+
  labs(title="Model with the PDO index") +
  theme_classic()

plt_drift_with_ts <- autoplot(res_with,
                              component=c("drift")
                              ) +
  labs(title="Model with the PDO index") +
  theme_classic()
```

```r
plt_level_drift_with_ts <- plt_level_with_ts +
  plt_drift_with_ts +
  patchwork::plot_layout(ncol=1)

plot(plt_level_drift_with_ts)
```



Gray areas in the above graph show 95% confidence interval.

Time Series Cross-Validation (tsCV)

Time series cross-validation (tsCV) approach evaluates model performance based on out-of-sample prediction errors while respecting the temporal ordering of the data, and thus provides an additional, independent perspective on model adequacy.

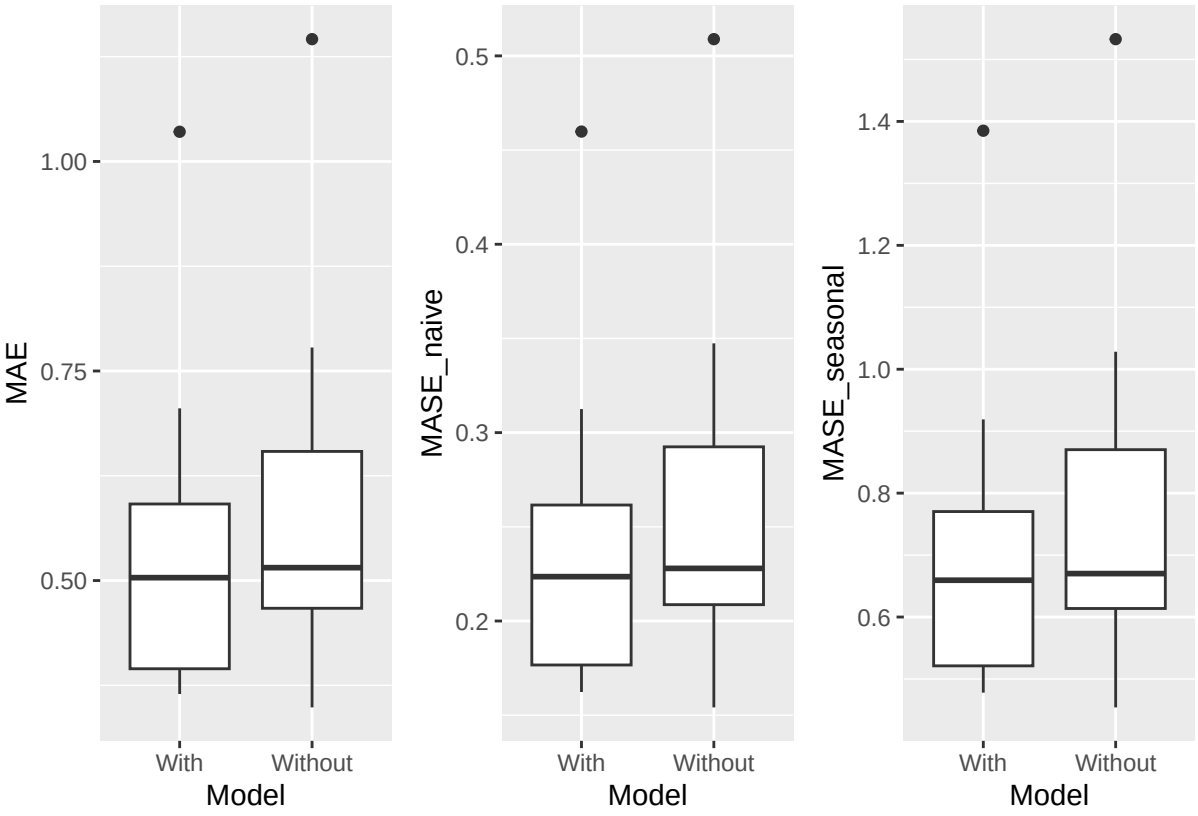
Time-series cross-validation is conducted by repeatedly fitting the model to an expanding training window and evaluating its predictive performance on subsequent observations. Unlike random cross-validation, this procedure avoids information leakage from the future to the past and is therefore well suited for temporal data.

By comparing cross-validation metrics for models with and without the exogenous PDO variable, we assess whether the improvement suggested by AIC is also reflected in out-of-sample predictive skill.

```r
# ts cross-validation of the model without exogenous variables

## Generate a list of training and test datasets with their indices
# Procedure for constructing year-based time-series cross-validation folds:
#
# First training data: January 1982-December 2017;
# First test data: one year starting from January 2018
#
```

```r
# Second training data: January 1983-December 2018;
# Second test data: one year starting from January 2019
# ...
#
# Eighth training data: January 1989-December 2024;
# Eighth test data: one year starting from January 2025
#
# These folds are automatically generated by the ts_train_test_split() function.

# Generate training and test dataset for model without PDO index
folds_without <- ts_train_test_split(
  temp_data = yamaguchi_sst_trim,
  exo_data = NULL,
  initial = 432, # 432 monthly observations from Jan 1982 to Dec 2017
  horizon = 12, # forecast 12 monthly time-series
  step = 12, # training data is moved by 12 months (1 years) steps
  fixed_window = TRUE,
  allow_partial = FALSE
  )

# Generate training and test dataset for model with PDO index
folds_with <- ts_train_test_split(
  temp_data = yamaguchi_sst_trim,
  exo_data = pdo_trim,
  initial = 432, # 432 monthly observations from Jan 1982 to Dec 2017
  horizon = 12, # forecast 12 monthly time-series
  step = 12, # training data is moved by 12 months (10 years) steps
  fixed_window = TRUE,
  allow_partial = FALSE
  )


# ****************************************
# Executing time-series cross validation

#-------------------------------------------------
# Model without the exogenous variable of PDA index
cv_without_results <- ts_cv_run(folds_without, ar_order = 2, use_season = TRUE)

# Couputing assessment indexes
metrics_without <- lapply(cv_without_results, compute_cv_metrics)

# tidy summary
cv_without_tbl <- ts_cv_collect(cv_without_results, metrics_without) %>%
  mutate(Model="Without")


#-------------------------------------------------
# Model with the exogenous variable of PDA index
cv_with_results <- ts_cv_run(folds_with, ar_order = 2, use_season = TRUE)

# Couputing assessment indexes
metrics_with <- lapply(cv_with_results, compute_cv_metrics)
```

```r
# tidy summary
cv_with_tbl <- ts_cv_collect(cv_with_results, metrics_with) %>%
  mutate(Model="With")
#}

cv_tbl <- bind_rows(cv_without_tbl, cv_with_tbl)


plt_MAE <- ggplot(data=cv_tbl,
                  aes(x=Model,y=MAE)) +
  geom_boxplot()

plt_MASE_naive <- ggplot(data=cv_tbl,
                   aes(x=Model,y=MASE_naive)) +
  geom_boxplot()

plt_MASE_seasonal <- ggplot(data=cv_tbl,
                   aes(x=Model,y=MASE_seasonal)) +
  geom_boxplot()


plt_tsCV <- plt_MAE + plt_MASE_naive + plt_MASE_seasonal + patchwork::plot_layout(nrow=1)

plot(plt_tsCV)
```



Analysis of tsCV shows that the model with the exogenous variable is better than the model without one. In this tutorial, the number of tsCV iterations has been set to eigth to reduce execution time. Please ensure you

perform a sufficient number of iterations during actual validation.

Taken together, the results consistently support the inclusion of the PDO index as an exogenous variable in the state-space model. The PDO coefficient is statistically significant, indicating a clear local effect on temperature variability, while the improvement in AIC demonstrates enhanced overall model fit. Furthermore, time-series cross-validation confirms that this improvement translates into superior out-of-sample predictive performance.

These complementary lines of evidence provide a robust and multifaceted justification for adopting the exogenous-variable model.

Using your own data

Quick example

`my_ts` is a user-supplied `ts` object. You can pass it directly:

```
## example of ts object (not run)
## my_ts <- ts(rnorm(100), frequency = 12)
res <- tempssm(my_ts)
```

Preparing your own dataset

CSV format

Prepare a CSV fine of monthly temperature time series with the following columns: - Year - Month - Temp

Example:

```
Year,Month,Temp
2010,8,13.6
2010,9,6.8
2010,10,NA
2010,11,-1.4
...
```

- Use NA for missing temperature values, and always keep the corresponding Year and Month entries.
- The SCV file must be comma-separated and UTR-8 encoded.

Convert CSV to a time series

An included example CSV file is available inst/extdata. The example dataset contains monthly temperature observations from Mt. Akadake, Hokkaido, Japan (1,840 m). The data originate from the Monitoring Sites 1000 Project by the Ministry of the Environment of Japan (KOZ01.zip, downloaded from https://www.biodic.go.jp/moni1000/findings/data/index.html).

```
path <- system.file("extdata", "example_monthly_temp.csv", package = "tempssm")
akadake_temp <- tempssm::read_monthly_temp_ts(path)
head(akadake_temp)
```

```
##        Jan Feb Mar Apr May Jun Jul   Aug   Sep   Oct   Nov   Dec
## 2010                                13.6   6.8   0.2  -6.8 -12.5
## 2011 -18.8
```

The function of `read_monthly_temp_ts()` internally uses the base R function `ts()` to create a time series object. You may alternatively construct a ts object manually if you need finer control.

References

The statistical modeling framework implemented in `tempssm` is based on the methodology described in Baba et al. (2024), with implementation details adapted from the accompanying supplementary materials and code repository.

Baba, S., Ishii, H., and Yoshiyama, T. (2024). Estimating sea temperature trends using a linear Gaussian state-space model in Jogashima, Kanagawa, Japan. *Bulletin of the Japanese Society of Fisheries Oceanography*, 88(3), 190–199. (In Japanese with an English abstract.) https://doi.org/10.34423/jsfo.88.3_190

Baba, S. (2024). Supplementary code and test data for estimating sea temperature trends using a linear Gaussian state-space model. GitHub repository: https://github.com/logics-of-blue/sea-temperature-trend-jogashima

Helske, J. (2017). KFAS: Exponential Family State Space Models in R.
Journal of Statistical Software, 78(10), 1–39. https://doi.org/10.18637/jss.v078.i10

Appendix: Utility Functions

Utility function: `read_monthly_temp_ts()`

The function `read_monthly_temp_ts()` converts monthly temperature data stored in a CSV file into an R `ts` object. It is intended to facilitate the ingestion of externally prepared time-series data into tempssm by enforcing a simple and consistent data format.

Example

For monthly temperature data, prepare a CSV file with a header row. By default, the column names must be `Year`, `Month`, and `Temp`, where `Temp` represents the observed temperature value.

```
Year,Month,Temp
2001,1,10.4
2001,2,8.2
2001,3,NA
2001,4,13.6
2001,5,16.1
...
```

- Use NA for missing temperature values, and always keep the corresponding Year and Month entries.
- The CSV file must be comma-separated and UTF-8 encoded.

The following example uses a sample CSV file included in the package.

```
tmp_csv <- tempfile(fileext = ".csv")
writeLines(
  c("Year,Month,Temp",
    "2001,1,10.4",
    "2001,2,8.2",
    "2001,3,NA",
    "2001,4,13.6",
    "2001,5,16.1"),
   tmp_csv
 )

# Read the CSV file and convert to a monthly ts object
temp_ts <- read_monthly_temp_ts(tmp_csv)
```

Utility function: `convert_monthly_df_to_ts()`

The function `convert_monthly_df_to_ts()` converts a data frame containing monthly temperature data into an R `ts` object. It is designed to support workflows where temperature data are first imported or prepared as a data frame before being used for time-series analysis.

The input data frame is expected to contain at least two columns: a date column (`Date`) and a temperature column (`Temp`).

Example

```
# Create a data frame of monthly temperature data
df <- data.frame(
  Date = as.Date(c(
    "2001-01-01",
    "2001-02-01",
    "2001-03-01",
    "2001-04-01",
    "2001-05-01")
    ),
  Temp = c(10.4, 8.2, NA, 13.6, 16.1)
  )

# Convert to a monthly ts object
temp_ts <- convert_monthly_df_to_ts(df)
head(temp_ts)
```

```
##       Jan  Feb  Mar  Apr  May
## 2001 10.4  8.2   NA 13.6 16.1
```

Utility function: `get_jma_sst_ts()`

The function `get_jma_sst_ts()` downloads publicly available daily mean sea-surface temperature (SST) data for Japanese coastal waters provided by the Japan Meteorological Agency (JMA). It aggregates the daily values into monthly means and returns the resulting time series as an object of class `ts`.

Example

The following example demonstrates how to download SST data for the southern coastal waters of Ibaraki Prefecture, Japan.

The argument `sea_area_id` specifies the numeric identifier of a sea area defined by JMA. For example, `138` corresponds to the coastal waters off southern Ibaraki. A list of available sea area IDs and their corresponding regions is provided by JMA at:

https://www.data.jma.go.jp/kaiyou/data/db/kaikyo/series/engan/eg_areano.html

```
sst_138_ts <- get_jma_sst_ts(sea_area_id = 138)
head(sst_138_ts)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1982 15.04419 14.22500 13.63903 15.31933 17.52258 19.52300
```

Utility function: `compute_temp_anomaly()`

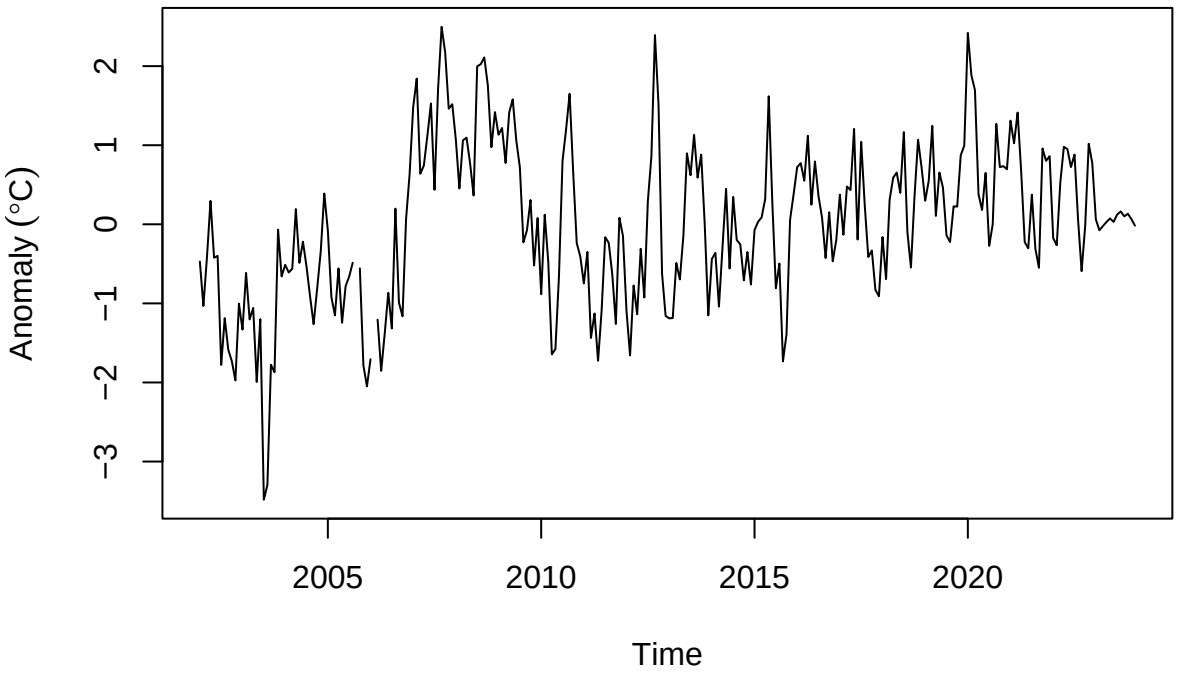
The function `compute_temp_anomaly()` computes monthly anomalies from a time series provided as an R `ts` object. Monthly anomalies are calculated by subtracting the corresponding long-term monthly mean from each observation, thereby removing the seasonal cycle while preserving interannual variability.

The reference period used to compute the monthly climatology can be specified via a function argument; by default, the climatology is computed over the entire available time series (see `?compute_temp_anomaly`).

This transformation is useful for exploratory analysis and modeling applications that focus on departures from typical seasonal conditions.

Example

```
# Generate temperature anomalies
data(niigata_sst)
niigata_sst_anomaly <- compute_temp_anomaly(niigata_sst)
plot(niigata_sst_anomaly, ylab=expression(Anomaly~(degree*C)))
```



Utility function: `compute_monthly_climatology()`

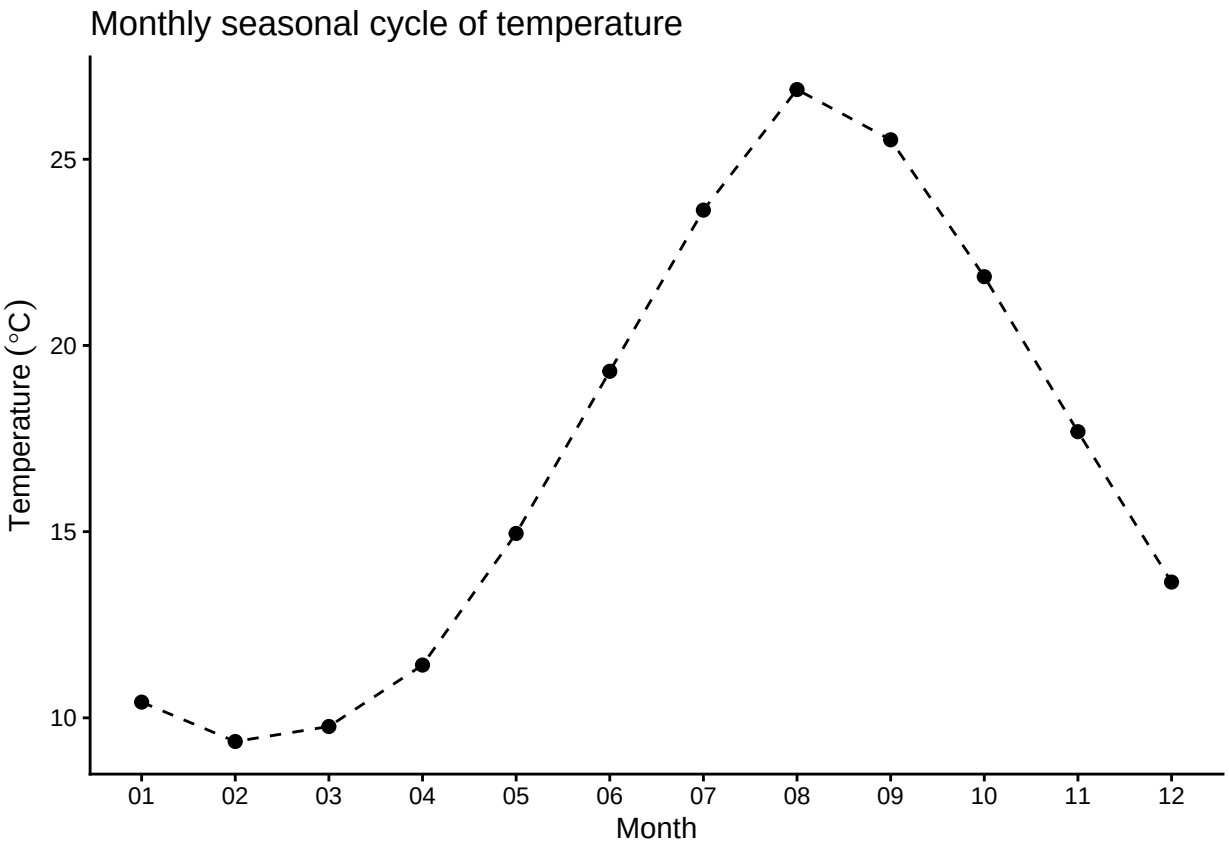
The function `compute_monthly_climatology()` computes the climatological mean seasonal cycle from a `ts` object by averaging each seasonal value across years. It is primarily intended for exploratory analysis and visualization of the seasonal structure in temperature time series.

Example

```
data(niigata_sst)
monthly_seasonal_cycle_niigata_sst <- compute_monthly_climatology(niigata_sst)
summary(monthly_seasonal_cycle_niigata_sst)
```

```
##      Month        Temperature
##  Min.   : 1.00   Min.   : 9.365
##  1st Qu.: 3.75   1st Qu.:11.169
##  Median : 6.50   Median :16.318
##  Mean   : 6.50   Mean   :17.036
##  3rd Qu.: 9.25   3rd Qu.:22.294
##  Max.   :12.00   Max.   :26.873
```

```r
plt_monthly_seasonal_cycle_niigata_sst <- ggplot(
  data = monthly_seasonal_cycle_niigata_sst,
  aes(x = Month, y = Temperature)
) +
  geom_point(size = 2) +
  geom_line(linetype = "dashed") +
  labs(
    title = "Monthly seasonal cycle of temperature",
    x = "Month",
    y = expression(Temperature~(degree*C))
  ) +
  scale_x_continuous(
    breaks = 1:12,
    labels = sprintf("%02d", 1:12)
  ) +
  theme_classic()


plot(plt_monthly_seasonal_cycle_niigata_sst)
```



These utility functions are provided to support data preparation and exploratory analysis and are not required for the core modeling functionality of tempssm.